Mounts Google Drive to the Colab environment, installs required external packages (such as `pgmpy` and `optuna`), imports standard machine learning libraries (like `pandas`, `numpy`, `scikit-learn`, and `xgboost`), and loads the raw credit card fraud dataset into a Pandas DataFrame

```
from google.colab import drive
drive.mount('/content/drive')
path_to_data = '/content/drive/MyDrive/Colab Notebooks/data/'
```

```
Mounted at /content/drive
```

```
import pandas as pd
import numpy as np
import pymc as pm
import arviz as az
import matplotlib.pyplot as plt
import seaborn as sns

from pgmpy.base import DAG
import networkx as nx

import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_va
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    roc_auc_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    precision_recall_curve
)

from imblearn.over_sampling import SMOTE

import xgboost as xgb

import tensorflow as tf
import tensorflow_probability as tfp
```

```
import optuna

df = pd.read_csv(path_to_data + 'fraudTest.csv')
df
```

	Unnamed: 0	trans_date_trans_time	cc_num	merchant	
0	0	2020-06-21 12:14:25	2291163933867244	fraud_Kirlin and Sons	pers
1	1	2020-06-21 12:14:33	3573030041201292	fraud_Sporer-Keebler	pers
2	2	2020-06-21 12:14:53	3598215285024754	fraud_Swaniawski, Nitzsche and Welch	hea
3	3	2020-06-21 12:15:15	3591919803438423	fraud_Haley Group	
4	4	2020-06-21 12:15:17	3526826139003047	fraud_Johnston-Casper	
...	...	...	...	...	
555714	555714	2020-12-31 23:59:07	30560609640617	fraud_Reilly and Sons	hea
555715	555715	2020-12-31 23:59:09	3556613125071656	fraud_Hoppe-Parisian	
555716	555716	2020-12-31 23:59:15	6011724471098086	fraud_Rau-Robel	
555717	555717	2020-12-31 23:59:24	4079773899158	fraud_Breitenberg LLC	
555718	555718	2020-12-31 23:59:34	4170689372027579	fraud_Dare-Marvin	ent

555719 rows × 23 columns

Configures academic-style plotting themes and generates key data visualizations for EDA. Specifically, it creates a log-scale bar chart to display extreme class imbalance, a boxplot to compare transaction amounts between legitimate and fraudulent classes, and a bar plot showing the fraud rate based on the hour of the day

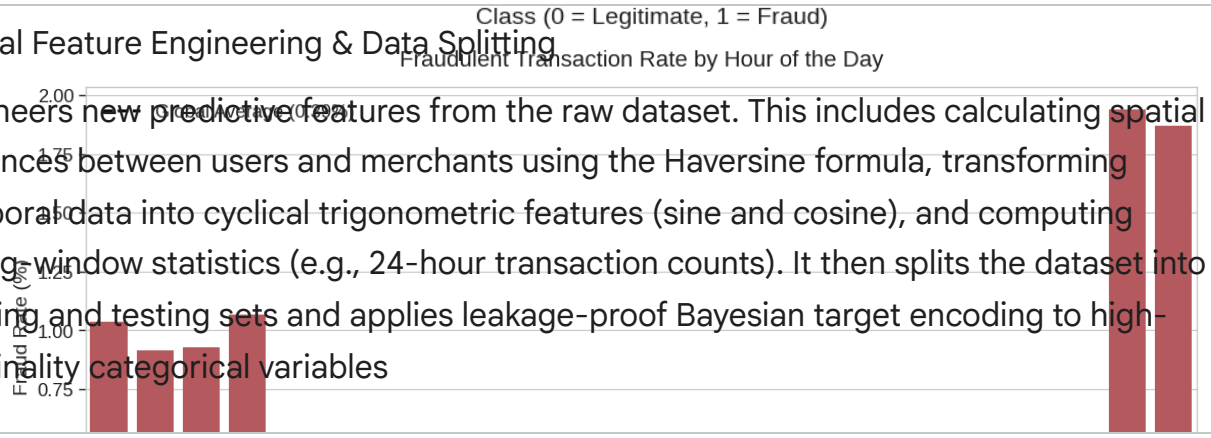
```
# Set academic-style plotting theme
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_context("paper", font_scale=1.5)

print("==== Generating Dataset Visualizations ====")

# Ensure the timestamp column is in datetime format for time-based analysis
if not pd.api.types.is_datetime64_any_dtype(df['trans_date_trans_time']):
    df['trans_date_trans_time'] = pd.to_datetime(df['trans_date_trans_time'])

# Extract hour from timestamp
df['hour'] = df['trans_date_trans_time'].dt.hour

# ----------------------------------------------------------
# Figure 1: Severe Class Imbalance
# ----------------------------------------------------------
plt.figure(figsize=(8, 6))

# Use log scale due to extreme imbalance to make minority class visible
ax = sns.countplot(data=df, x='is_fraud', palette=['#4C72B0', '#C44E52'])
plt.yscale('log')

plt.title('Log-Scale Distribution of Transaction Classes', fontsize=16, pad=15)
plt.xlabel('Class (0 = Legitimate, 1 = Fraud)', fontsize=14)
plt.ylabel('Count (Log Scale)', fontsize=14)

# Add count labels on top of bars
for p in ax.patches:
    ax.annotate(format(p.get_height(), '.0f'),
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center',
                xytext=(0, 9),
                textcoords='offset points')

plt.tight_layout()
plt.savefig('fig_class_imbalance.png', dpi=300)
plt.show()

# ----------------------------------------------------------
# Figure 2: Transaction Amount Distribution by Class
# ----------------------------------------------------------
plt.figure(figsize=(10, 6))
```

```
# Hide extreme outliers to better visualize the main distribution
sns.boxplot(data=df, x='is_fraud', y='amt',
            palette=['#4C72B0', '#C44E52'],
            showfliers=False)

plt.title('Transaction Amount Distribution by Class', fontsize=16, pad=15)
plt.xlabel('Class (0 = Legitimate, 1 = Fraud)', fontsize=14)
plt.ylabel('Transaction Amount ($)', fontsize=14)

plt.tight_layout()
plt.savefig('fig_amount_distribution.png', dpi=300)
plt.show()

# ----------------------------------------------------------
# Figure 3: Fraud Rate by Hour
# ----------------------------------------------------------

# Compute fraud rate per hour (fraud count / total transactions per hour)
hourly_fraud_rate = df.groupby('hour')['is_fraud'].mean() * 100

plt.figure(figsize=(12, 6))

# Bar plot showing hourly fraud rate
sns.barplot(x=hourly_fraud_rate.index,
            y=hourly_fraud_rate.values,
            color='#C44E52')

plt.title('Fraudulent Transaction Rate by Hour of the Day', fontsize=16, pad=15
plt.xlabel('Hour of the Day (0–23)', fontsize=14)
plt.ylabel('Fraud Rate (%)', fontsize=14)

# Add global average fraud rate as a baseline reference line
global_mean = df['is_fraud'].mean() * 100
plt.axhline(global_mean,
            color='black',
            linestyle='--',
            linewidth=2,
            label=f'Global Average ({global_mean:.2f}%)')

plt.legend()

plt.tight_layout()
plt.savefig('fig_hourly_fraud.png', dpi=300)
plt.show()

print("Visualization complete! Saved as fig_class_imbalance.png, fig_amount_dis
```

```
==== Generating Dataset Visualizations ====
/tmp/ipykernel_1237/1082513716.py:20: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in

  ax = sns.countplot(data=df, x='is_fraud', palette=['#4C72B0', '#C44E52'])
```

```
/tmp/ipykernel_1237/1082513716.py:45: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in

  sns.boxplot(data=df, x='is_fraud', y='amt',
```

Global Feature Engineering & Data Splitting



Engineers new predictive features from the raw dataset. This includes calculating spatial distances between users and merchants using the Haversine formula, transforming temporal data into cyclical trigonometric features (sine and cosine), and computing rolling-window statistics (e.g., 24-hour transaction counts). It then splits the dataset into training and testing sets and applies leakage-proof Bayesian target encoding to high-cardinality categorical variables

```
# ==========================================
# Helper Function: Calculate Haversine Distance
# ==========================================
def haversine_distance(lat1, lon1, lat2, lon2):
    R = 6371.0 # Earth radius in kilometers
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2)**2
    c = 2 * np.arcsin(np.sqrt(a))
    return R * c

print("==== Stage 1: Global Feature Engineering (Pre-Split) ====\n")

# Drop the 'Unnamed: 0' index column if it exists
if 'Unnamed: 0' in df.columns:
    df = df.drop(columns=['Unnamed: 0'])

# Sort by time FIRST to ensure time differences and rolling windows are calcula
df['trans_date_trans_time'] = pd.to_datetime(df['trans_date_trans_time'])
df = df.sort_values(by='trans_date_trans_time').reset_index(drop=True)

df_feat = df.copy()

# ----------------------------------------------------
# A. Basic & Demographic Features
# ----------------------------------------------------
df_feat['dob'] = pd.to_datetime(df_feat['dob'])
df_feat['age'] = (df_feat['trans_date_trans_time'] - df_feat['dob']).dt.days //
df_feat['gender_bin'] = df_feat['gender'].map({'M': 1, 'F': 0})

# ----------------------------------------------------
# B. Geographic & Spatial Features
# ----------------------------------------------------
df_feat['distance_km'] = haversine_distance(
    df_feat['lat'], df_feat['long'],
    df_feat['merch_lat'], df_feat['merch_long']
```

```
)

# ----------------------------------------------------
# C. Temporal & Cyclical Features
# ----------------------------------------------------
df_feat['hour'] = df_feat['trans_date_trans_time'].dt.hour
df_feat['day_of_week'] = df_feat['trans_date_trans_time'].dt.dayofweek

df_feat['hour_cos'] = np.cos(2 * np.pi * df_feat['hour'] / 24)
df_feat['hour_sin'] = np.sin(2 * np.pi * df_feat['hour'] / 24)
df_feat['day_cos'] = np.cos(2 * np.pi * df_feat['day_of_week'] / 7)
df_feat['day_sin'] = np.sin(2 * np.pi * df_feat['day_of_week'] / 7)

# Calculate time difference in seconds since the last transaction for each carc
# Because the dataframe is whole and sorted, no history is broken here!
df_feat = df_feat.sort_values(by=['cc_num', 'trans_date_trans_time'])
df_feat['time_diff'] = df_feat.groupby('cc_num')['trans_date_trans_time'].diff(

# ----------------------------------------------------
# D. Interaction & Rolling Window Features
# ----------------------------------------------------
df_feat = df_feat.set_index('trans_date_trans_time')

# Rolling metrics across the full history
df_feat['tx_count_24h'] = df_feat.groupby('cc_num')['amt'].transform(lambda x:
df_feat['amt_sum_24h'] = df_feat.groupby('cc_num')['amt'].transform(lambda x: x
df_feat['amt_7d_mean'] = df_feat.groupby('cc_num')['amt'].transform(lambda x: x
df_feat['tx_count'] = df_feat.groupby('cc_num').cumcount() + 1

df_feat = df_feat.reset_index()

# Amount anomaly ratio
safe_mean = df_feat['amt_7d_mean'].replace(0, 0.01)
df_feat['amt_anomaly_ratio'] = df_feat['amt'] / safe_mean
df_feat['amt_x_hour'] = df_feat['amt'] * df_feat['hour_cos']

# One-Hot Encoding for categories
df_feat = pd.get_dummies(df_feat, columns=['category'], prefix='cat')


print("==== Stage 2: Split ====\n")
# ==========================================
# Perform Random Split (80% Train, 20% Test)
# ==========================================
train_df_engineered, test_df_engineered = train_test_split(df_feat, test_size=0

# Reset indices for clean dataframes
train_df_engineered = train_df_engineered.reset_index(drop=True)
test_df_engineered = test_df_engineered.reset_index(drop=True)

print(f"Train size: {len(train_df_engineered)}, Test size: {len(test_df_enginee
```

```
print("\n==== Stage 3: Leakage-Proof Target Encoding ====\n")
# ==========================================
# E. Target Encoding (Post-Split)
# ==========================================
# Strict leakage prevention: Calculate risk maps ONLY on the training set
job_risk_map = train_df_engineered.groupby('job')['is_fraud'].mean().to_dict()
merchant_risk_map = train_df_engineered.groupby('merchant')['is_fraud'].mean().
global_mean_fraud = train_df_engineered['is_fraud'].mean()

# Apply the learned mappings to the Training Set
train_df_engineered['job_encoded'] = train_df_engineered['job'].map(job_risk_ma
train_df_engineered['merchant_encoded'] = train_df_engineered['merchant'].map(m

# Apply the SAME learned mappings to the Test Set (simulate unseen data)
test_df_engineered['job_encoded'] = test_df_engineered['job'].map(job_risk_map)
test_df_engineered['merchant_encoded'] = test_df_engineered['merchant'].map(mer

print("Feature engineering complete! Both sets are ready for modeling.")
```

```
==== Stage 1: Global Feature Engineering (Pre-Split) ====

==== Stage 2: Split ====

Train size: 444575, Test size: 111144

==== Stage 3: Leakage-Proof Target Encoding ====

Feature engineering complete! Both sets are ready for modeling.
```

Feature Selection via LASSO Regression

Drops non-predictive columns, scales the features using `StandardScaler`, and trains a Logistic Regression model with L1 regularization (LASSO). [cite_start]This technique automatically prunes redundant or irrelevant features by shrinking their coefficients to exactly zero, leaving only the most important predictors.

```
print("Starting LASSO-based feature selection...")

# ==========================================
# 1. Data Preparation (Cleaning & Splitting)
# ==========================================
# Drop non-numeric, raw text, ID columns, and target variable
columns_to_drop = [
    'is_fraud', 'trans_date_trans_time', 'cc_num', 'merchant', 'category',
    'first', 'last', 'gender', 'street', 'city', 'state', 'zip',
    'job', 'dob', 'trans_num', 'unix_time',
```

```
    'lat', 'long', 'merch_lat', 'merch_long'  # replaced by distance_km
]

# Only drop columns that actually exist
drop_cols = [col for col in columns_to_drop if col in train_df_engineered.colum

# Feature matrix and target
X_train = train_df_engineered.drop(columns=drop_cols)
y_train = train_df_engineered['is_fraud']

# Handle missing values (e.g., first transaction time_diff)
X_train = X_train.fillna(0)

# ==========================================
# 2. Feature Scaling (MANDATORY for LASSO)
# ==========================================
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Convert back to DataFrame to preserve column names
X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X_train.columns)

# ==========================================
# 3. Train LASSO Logistic Regression Model
# ==========================================
# Key parameters:
# penalty='l1' -> enables LASSO regularization
# solver='liblinear' -> supports L1 penalty
# C=0.05 -> inverse of regularization strength (smaller = stronger shrinkage)

print("Fitting LASSO model (this may take a few minutes)...")

lasso_model = LogisticRegression(
    penalty='l1',
    solver='liblinear',
    C=0.01,
    random_state=42,
    max_iter=1000
)

lasso_model.fit(X_train_scaled_df, y_train)

# ==========================================
# 4. Extract Coefficients & Build Feature Funnel
# ==========================================
coefficients = lasso_model.coef_[0]

lasso_results = pd.DataFrame({
    'Feature': X_train.columns,
    'Coefficient': coefficients,
    'Absolute_Coefficient': np.abs(coefficients)
```

```
})

# Separate retained vs pruned features
retained_features = lasso_results[
    lasso_results['Coefficient'] != 0
].sort_values(by='Absolute_Coefficient', ascending=False)

pruned_features = lasso_results[
    lasso_results['Coefficient'] == 0
]

# ==========================================
# 5. Display Results
# ==========================================
print("\n" + "=" * 40)
print(f"LASSO feature selection completed!")
print(f"Original number of features: {len(X_train.columns)}")
print(f"Retained features (non-zero coefficients): {len(retained_features)}")
print(f"Pruned features (shrunk to zero): {len(pruned_features)}")
print("=" * 40)

print("\nTop 10 Important Features:")
print(retained_features.head(10)[['Feature', 'Coefficient']])

print("\nExample of Removed (Zeroed) Features:")
print(pruned_features.head(10)['Feature'].tolist())

# Save selected feature names for downstream modeling
lasso_selected_columns = retained_features['Feature'].tolist()
```

```
Starting LASSO-based feature selection...
Fitting LASSO model (this may take a few minutes)...

========================================
LASSO feature selection completed!
Original number of features: 34
Retained features (non-zero coefficients): 18
Pruned features (shrunk to zero): 16
========================================

Top 10 Important Features:
              Feature  Coefficient
7            hour_cos     0.998470
32        job_encoded     0.774438
33   merchant_encoded     0.478185
15           tx_count    -0.404861
13        amt_sum_24h     0.385709
9             day_cos    -0.207911
14        amt_7d_mean     0.203120
22    cat_grocery_pos     0.192903
2                hour     0.191468
16  amt_anomaly_ratio     0.161771
```

```
Example of Removed (Zeroed) Features:
['age', 'gender_bin', 'distance_km', 'hour_sin', 'day_sin', 'tx_count_24h', 'cat
```

Multicollinearity Check via VIF

```
print("Starting multicollinearity check and manual pruning...")

# 1. Get the features retained by LASSO
features_from_lasso = retained_features['Feature'].tolist()
X_retained = X_train_scaled_df[features_from_lasso]

# 2. Calculate the current VIF (Variance Inflation Factor)
# VIF > 5 or 10 usually indicates high multicollinearity
vif_data = pd.DataFrame()
vif_data["Feature"] = X_retained.columns
vif_data["VIF"] = [variance_inflation_factor(X_retained.values, i) for i in rar
vif_data = vif_data.sort_values(by="VIF", ascending=False)

print("==== VIF Multicollinearity Check after LASSO ====")
print(vif_data.head(10))


# 4. Generate the final feature pool for Bayesian HMC
hmc_candidate_features = [f for f in features_from_lasso]

print("\n==== Final Candidate Features for Bayesian HMC ====")
print(hmc_candidate_features)
print(f"Total features remaining: {len(hmc_candidate_features)}")
```

```
Starting multicollinearity check and manual pruning...
==== VIF Multicollinearity Check after LASSO ====
              Feature       VIF
16                amt  3.259372
9   amt_anomaly_ratio  2.975143
4         amt_sum_24h  1.559325
8                hour  1.539777
7     cat_grocery_pos  1.482529
2    merchant_encoded  1.411862
6         amt_7d_mean  1.363652
14  cat_gas_transport  1.311569
17       cat_misc_net  1.241945
5             day_cos  1.237650

==== Final Candidate Features for Bayesian HMC ====
['hour_cos', 'job_encoded', 'merchant_encoded', 'tx_count', 'amt_sum_24h', 'day_
Total features remaining: 18
```

Bayesian Posterior Estimation & Statistical Pruning

```
print("Preparing data for Bayesian HMC modeling (with stabilized sampling)...")

# ==========================================
# 1. Feature Selection (Using the 13 candidate features)
# ==========================================
hmc_features = ['hour_cos', 'job_encoded', 'merchant_encoded', 'amt_sum_24h',
                'amt_anomaly_ratio', 'city_pop', 'time_diff', 'amt_x_hour',
                'day_of_week', 'amt', 'gender_bin', 'tx_count_24h', 'age']


X_hmc_full = X_train_scaled_df[hmc_features]
y_hmc_full = y_train.reset_index(drop=True)

# ==========================================
# 2. Stratified Subsampling (MODIFICATION 1 APPLIED)
# ==========================================
# CRITICAL: Lock the NumPy random seed to ensure exact same data is drawn every
np.random.seed(42)

# Increased sample size to 10000 for more stable posterior estimates
sample_size = 10000
fraud_ratio = y_hmc_full.mean()

fraud_indices = y_hmc_full[y_hmc_full == 1].index
normal_indices = y_hmc_full[y_hmc_full == 0].index

n_fraud = int(sample_size * fraud_ratio)
n_normal = sample_size - n_fraud

sampled_fraud = np.random.choice(fraud_indices, n_fraud, replace=False)
sampled_normal = np.random.choice(normal_indices, n_normal, replace=False)

final_indices = np.concatenate([sampled_fraud, sampled_normal])

X_hmc = X_hmc_full.iloc[final_indices].values
y_hmc = y_hmc_full.iloc[final_indices].values

print(f"Data successfully downsampled to {sample_size} rows with fixed random s

# ==========================================
# 3. Build and Sample the Bayesian HMC Model (MODIFICATION 3 APPLIED)
# ==========================================
print("\nBuilding PyMC model and starting NUTS (HMC) sampling...")
print("This may take a bit longer due to increased tuning steps. Please wait...

with pm.Model() as bayesian_model:
    intercept = pm.Normal('Intercept', mu=0, sigma=1.5)
    weights = pm.Normal('Weights', mu=0, sigma=1.5, shape=len(hmc_features))
```

```
        mu = intercept + pm.math.dot(X_hmc, weights)
        p = pm.math.invlogit(mu)
        y_obs = pm.Bernoulli('y_obs', p=p, observed=y_hmc)

        # CRITICAL: Increased tuning steps, draws, and target_accept for absolute s
        trace = pm.sample(
            draws=1000,
            tune=2000,
            chains=4,
            cores=1,
            target_accept=0.95,  # Increased from 0.90 to reduce sampling divergenc
            random_seed=42,      # Locks PyMC internal sampling engine
            progressbar=True
        )

    # ==========================================
    # 4. Extract Results and Check for Zero-Crossing (85% HDI)
    # ==========================================
    print("\n==== HMC Sampling Complete! Analyzing Posterior Statistics ====")

    summary_df = az.summary(trace, var_names=['Weights'], hdi_prob=0.85)
    summary_df.index = hmc_features

    print("\nBayesian Posterior 85% Credible Intervals:")
    print(summary_df[['mean', 'sd', 'hdi_7.5%', 'hdi_92.5%']])

    print("\n Evaluating Statistical Uncertainty (Zero-Crossing Check at 85%):")
    features_to_keep = []
    features_to_prune = []

    for feature in hmc_features:
        lower_bound = summary_df.loc[feature, 'hdi_7.5%']
        upper_bound = summary_df.loc[feature, 'hdi_92.5%']

        if lower_bound < 0 and upper_bound > 0:
            print(f" PRUNE: [{feature}] crosses ZERO ({lower_bound:.2f} to {upper_b
            features_to_prune.append(feature)
        else:
            sign = "Positive" if lower_bound > 0 else "Negative"
            print(f" KEEP: [{feature}] is a strong {sign} predictor.")
            features_to_keep.append(feature)

    print(f"\nFinal retained features after stabilized 85% HMC: {features_to_keep}"

    # ==========================================
    # 5. Generate the Forest Plot (Visual Evidence)
    # ==========================================
    plt.figure(figsize=(12, 7))

    ax = az.plot_forest(
        trace,
```

```
        var_names=['Weights'],
        hdi_prob=0.85,
        combined=True,
        figsize=(10, 6),
        textsize=12
    )

    ax[0].set_yticklabels(hmc_features[::-1])
    ax[0].axvline(0, color='red', linestyle='--', linewidth=2, alpha=0.8)
    ax[0].set_title('Bayesian HMC Posterior Estimates (85% HDI - Stabilized)', font
    ax[0].set_xlabel('Log-Odds Coefficient Value', fontsize=12)

    plt.tight_layout()
    plt.show()
```

```
Preparing data for Bayesian HMC modeling (with stabilized sampling)...
Data successfully downsampled to 10000 rows with fixed random seed.

Building PyMC model and starting NUTS (HMC) sampling...
This may take a bit longer due to increased tuning steps. Please wait...
```

Progress	Draw	Divergences	Step size	Grad evals
━━━━━━━━━━━━━━━━━━━━━━━━━━━━	3000	0	0.231	31
━━━━━━━━━━━━━━━━━━━━━━━━━━━━	3000	0	0.202	31
━━━━━━━━━━━━━━━━━━━━━━━━━━━━	3000	0	0.243	15
━━━━━━━━━━━━━━━━━━━━━━━━━━━━	3000	0	0.205	31

```
==== HMC Sampling Complete! Analyzing Posterior Statistics ====

Bayesian Posterior 85% Credible Intervals:
                    mean     sd  hdi_7.5%  hdi_92.5%
hour_cos           0.863  0.273     0.481      1.254
job_encoded        0.778  0.158     0.563      1.015
merchant_encoded   0.553  0.125     0.379      0.729
amt_sum_24h        0.594  0.079     0.474      0.701
amt_anomaly_ratio  0.168  0.168    -0.059      0.418
city_pop          -1.023  0.423    -1.561     -0.364
time_diff         -0.198  0.246    -0.529      0.166
amt_x_hour         0.187  0.076     0.074      0.290
```

```
print("Preparing data for Bayesian HMC modeling (with stabilized sampling)...")

# ==========================================
# 1. Feature Selection (Using the 7 candidate features)
# ==========================================
hmc_features = ['hour_cos', 'job_encoded', 'merchant_encoded', 'amt_sum_24h',
                'city_pop', 'amt', 'tx_count_24h']


X_hmc_full = X_train_scaled_df[hmc_features]
y_hmc_full = y_train.reset_index(drop=True)

# ==========================================
# 2. Stratified Subsampling (MODIFICATION 1 APPLIED)
# ==========================================
# CRITICAL: Lock the NumPy random seed to ensure exact same data is drawn every
np.random.seed(23)

# Increased sample size to 10000 for more stable posterior estimates
sample_size = 10000
fraud_ratio = y_hmc_full.mean()

fraud_indices = y_hmc_full[y_hmc_full == 1].index
normal_indices = y_hmc_full[y_hmc_full == 0].index
```

```
n_fraud = int(sample_size * fraud_ratio)
n_normal = sample_size - n_fraud

sampled_fraud = np.random.choice(fraud_indices, n_fraud, replace=False)
sampled_normal = np.random.choice(normal_indices, n_normal, replace=False)

final_indices = np.concatenate([sampled_fraud, sampled_normal])

X_hmc = X_hmc_full.iloc[final_indices].values
y_hmc = y_hmc_full.iloc[final_indices].values

print(f"Data successfully downsampled to {sample_size} rows with fixed random s

# ==========================================
# 3. Build and Sample the Bayesian HMC Model (MODIFICATION 3 APPLIED)
# ==========================================
print("\nBuilding PyMC model and starting NUTS (HMC) sampling...")
print("This may take a bit longer due to increased tuning steps. Please wait...

with pm.Model() as bayesian_model:
    intercept = pm.Normal('Intercept', mu=0, sigma=1.5)
    weights = pm.Normal('Weights', mu=0, sigma=1.5, shape=len(hmc_features))

    mu = intercept + pm.math.dot(X_hmc, weights)
    p = pm.math.invlogit(mu)
    y_obs = pm.Bernoulli('y_obs', p=p, observed=y_hmc)

    # CRITICAL: Increased tuning steps, draws, and target_accept for absolute s
    trace = pm.sample(
        draws=1000,
        tune=2000,
        chains=4,
        cores=1,
        target_accept=0.95,  # Increased from 0.90 to reduce sampling divergenc
        random_seed=23,
        progressbar=True
    )

# ==========================================
# 4. Extract Results and Check for Zero-Crossing (85% HDI)
# ==========================================
print("\n==== HMC Sampling Complete! Analyzing Posterior Statistics ====")

summary_df = az.summary(trace, var_names=['Weights'], hdi_prob=0.85)
summary_df.index = hmc_features

print("\nBayesian Posterior 85% Credible Intervals:")
print(summary_df[['mean', 'sd', 'hdi_7.5%', 'hdi_92.5%']])

print("\n Evaluating Statistical Uncertainty (Zero-Crossing Check at 85%):")
```

```
features_to_keep = []
features_to_prune = []

for feature in hmc_features:
    lower_bound = summary_df.loc[feature, 'hdi_7.5%']
    upper_bound = summary_df.loc[feature, 'hdi_92.5%']

    if lower_bound < 0 and upper_bound > 0:
        print(f" PRUNE: [{feature}] crosses ZERO ({lower_bound:.2f} to {upper_b
        features_to_prune.append(feature)
    else:
        sign = "Positive" if lower_bound > 0 else "Negative"
        print(f" KEEP: [{feature}] is a strong {sign} predictor.")
        features_to_keep.append(feature)

print(f"\nFinal retained features after stabilized 85% HMC: {features_to_keep}"

# ==========================================
# 5. Generate the Forest Plot (Visual Evidence)
# ==========================================
plt.figure(figsize=(12, 7))

ax = az.plot_forest(
    trace,
    var_names=['Weights'],
    hdi_prob=0.85,
    combined=True,
    figsize=(10, 6),
    textsize=12
)

ax[0].set_yticklabels(hmc_features[::-1])
ax[0].axvline(0, color='red', linestyle='--', linewidth=2, alpha=0.8)
ax[0].set_title('Bayesian HMC Posterior Estimates (85% HDI - Stabilized)', font
ax[0].set_xlabel('Log-Odds Coefficient Value', fontsize=12)

plt.tight_layout()
plt.show()
```

```
Preparing data for Bayesian HMC modeling (with stabilized sampling)...
Data successfully downsampled to 10000 rows with fixed random seed.

Building PyMC model and starting NUTS (HMC) sampling...
This may take a bit longer due to increased tuning steps. Please wait...
```

Progress	Draw	Divergences	Step size	Grad evals
━━━━━━━━━━━━━━━━━━━━━━━━━━━━	3000	0	0.243	15
━━━━━━━━━━━━━━━━━━━━━━━━━━━━	3000	0	0.320	15
━━━━━━━━━━━━━━━━━━━━━━━━━━━━	3000	0	0.244	7
━━━━━━━━━━━━━━━━━━━━━━━━━━━━	3000	0	0.250	15

```
print("Preparing data for Bayesian HMC modeling (with stabilized sampling)...")

# ==========================================
# 1. Feature Selection (Using the 6 candidate features)
# ==========================================
hmc_features = ['hour_cos', 'job_encoded', 'merchant_encoded', 'amt_sum_24h',
                'amt', 'tx_count_24h']


X_hmc_full = X_train_scaled_df[hmc_features]
y_hmc_full = y_train.reset_index(drop=True)

# ==========================================
# 2. Stratified Subsampling (MODIFICATION 1 APPLIED)
# ==========================================
# CRITICAL: Lock the NumPy random seed to ensure exact same data is drawn every
np.random.seed(23)

# Increased sample size to 10000 for more stable posterior estimates
sample_size = 10000
fraud_ratio = y_hmc_full.mean()

fraud_indices = y_hmc_full[y_hmc_full == 1].index
normal_indices = y_hmc_full[y_hmc_full == 0].index

n_fraud = int(sample_size * fraud_ratio)
n_normal = sample_size - n_fraud

sampled_fraud = np.random.choice(fraud_indices, n_fraud, replace=False)
sampled_normal = np.random.choice(normal_indices, n_normal, replace=False)

final_indices = np.concatenate([sampled_fraud, sampled_normal])

X_hmc = X_hmc_full.iloc[final_indices].values
y_hmc = y_hmc_full.iloc[final_indices].values

print(f"Data successfully downsampled to {sample_size} rows with fixed random s
```

```
# ==========================================
# 3. Build and Sample the Bayesian HMC Model (MODIFICATION 3 APPLIED)
# ==========================================
print("\nBuilding PyMC model and starting NUTS (HMC) sampling...")
print("This may take a bit longer due to increased tuning steps. Please wait...

with pm.Model() as bayesian_model:
    intercept = pm.Normal('Intercept', mu=0, sigma=1.5)
    weights = pm.Normal('Weights', mu=0, sigma=1.5, shape=len(hmc_features))

    mu = intercept + pm.math.dot(X_hmc, weights)
    p = pm.math.invlogit(mu)
    y_obs = pm.Bernoulli('y_obs', p=p, observed=y_hmc)

    # CRITICAL: Increased tuning steps, draws, and target_accept for absolute s
    trace = pm.sample(
        draws=1000,
        tune=2000,
        chains=4,
        cores=1,
        target_accept=0.95,  # Increased from 0.90 to reduce sampling divergenc
        random_seed=23,
        progressbar=True
    )

# ==========================================
# 4. Extract Results and Check for Zero-Crossing (85% HDI)
# ==========================================
print("\n==== HMC Sampling Complete! Analyzing Posterior Statistics ====")

summary_df = az.summary(trace, var_names=['Weights'], hdi_prob=0.85)
summary_df.index = hmc_features

print("\nBayesian Posterior 85% Credible Intervals:")
print(summary_df[['mean', 'sd', 'hdi_7.5%', 'hdi_92.5%']])

print("\n Evaluating Statistical Uncertainty (Zero-Crossing Check at 85%):")
features_to_keep = []
features_to_prune = []

for feature in hmc_features:
    lower_bound = summary_df.loc[feature, 'hdi_7.5%']
    upper_bound = summary_df.loc[feature, 'hdi_92.5%']

    if lower_bound < 0 and upper_bound > 0:
        print(f" PRUNE: [{feature}] crosses ZERO ({lower_bound:.2f} to {upper_b
        features_to_prune.append(feature)
    else:
        sign = "Positive" if lower_bound > 0 else "Negative"
        print(f" KEEP: [{feature}] is a strong {sign} predictor.")
```

```
            features_to_keep.append(feature)

    print(f"\nFinal retained features after stabilized 85% HMC: {features_to_keep}"

    # ==========================================
    # 5. Generate the Forest Plot (Visual Evidence)
    # ==========================================
    plt.figure(figsize=(12, 7))

    ax = az.plot_forest(
        trace,
        var_names=['Weights'],
        hdi_prob=0.85,
        combined=True,
        figsize=(10, 6),
        textsize=12
    )

    ax[0].set_yticklabels(hmc_features[::-1])
    ax[0].axvline(0, color='red', linestyle='--', linewidth=2, alpha=0.8)
    ax[0].set_title('Bayesian HMC Posterior Estimates (85% HDI - Stabilized)', font
    ax[0].set_xlabel('Log-Odds Coefficient Value', fontsize=12)

    plt.tight_layout()
    plt.show()
```

```
Preparing data for Bayesian HMC modeling (with stabilized sampling)...
Data successfully downsampled to 10000 rows with fixed random seed.

Building PyMC model and starting NUTS (HMC) sampling...
This may take a bit longer due to increased tuning steps. Please wait...
```

Progress	Draw	Divergences	Step size	Grad evals
━━━━━━━━━━━━━━━━━━━━━━━━━━━━	3000	0	0.260	15
━━━━━━━━━━━━━━━━━━━━━━━━━━━━	3000	0	0.250	31
━━━━━━━━━━━━━━━━━━━━━━━━━━━━	3000	0	0.232	15
━━━━━━━━━━━━━━━━━━━━━━━━━━━━	3000	0	0.237	15

Baseline Classical Logistic Regression

```
print("==== Classic Logistic Regression ====\n")

# ==========================================
# 1. Data Preparation
# ==========================================
# Use the previously selected core production features
pruned_production_features = [
    'hour_cos', 'job_encoded', 'merchant_encoded',
    'amt_sum_24h', 'amt', 'tx_count_24h'
]

# Extract feature matrices and labels
X_train_raw = train_df_engineered[pruned_production_features].copy()
y_train = train_df_engineered['is_fraud'].values

X_test_raw = test_df_engineered[pruned_production_features].copy()
y_test = test_df_engineered['is_fraud'].values
amt_test_full = test_df_engineered['amt'].values

# Standardization (feature scaling)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_raw)
X_test_scaled = scaler.transform(X_test_raw)

# ==========================================
# 2. Train Classic Logistic Regression
# ==========================================
print("Training Classic Logistic Regression")

# Pure classical model: no sample weighting, treat all samples equally
classic_log_reg = LogisticRegression(max_iter=1000, random_state=42)
classic_log_reg.fit(X_train_scaled, y_train)

# ==========================================
# 3. Evaluate on Test Set
# ==========================================
```

```
y_pred_classic = classic_log_reg.predict(X_test_scaled)
y_prob_classic = classic_log_reg.predict_proba(X_test_scaled)[:, 1]

# Compute business-related metrics (financial fraud prevention impact)
total_fraud_mask = (y_test == 1)
total_fraud_amt = np.sum(amt_test_full[total_fraud_mask])

# True positives: correctly detected fraud transactions
tp_mask_classic = (y_pred_classic == 1) & (y_test == 1)

# Total protected fraud amount (successfully intercepted fraud)
protected_amt_classic = np.sum(amt_test_full[tp_mask_classic])

# Percentage of fraud amount successfully protected
protected_ratio_classic = (protected_amt_classic / total_fraud_amt) * 100 if tc

print("\n[Evaluation] Classic Logistic Regression Results:")
print(f"ROC-AUC:          {roc_auc_score(y_test, y_prob_classic):.4f}")
print(f"Precision:        {precision_score(y_test, y_pred_classic):.4f}")
print(f"Recall:           {recall_score(y_test, y_pred_classic):.4f}")
print(f"F1-Score:         {f1_score(y_test, y_pred_classic):.4f}")
print(f"Protected Ratio:  {protected_ratio_classic:.2f}%")
```

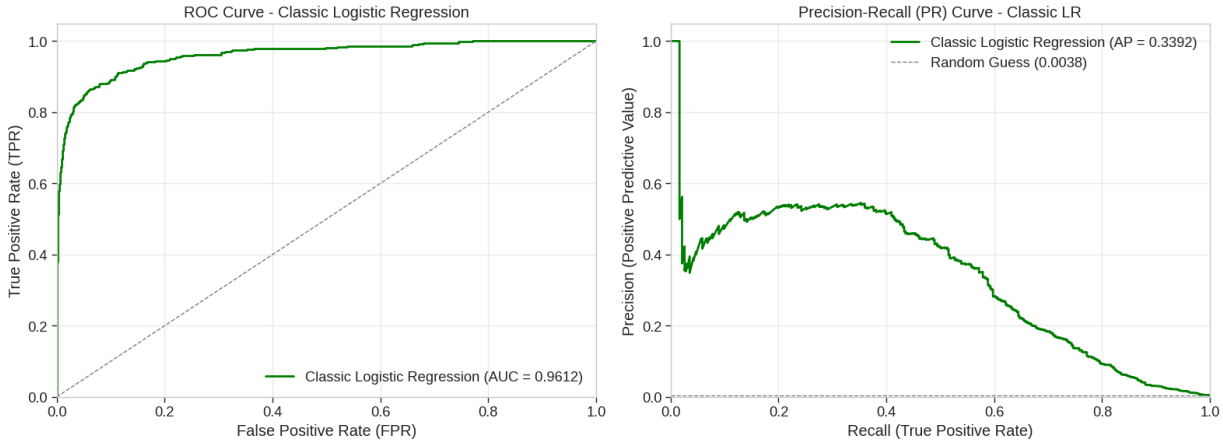
```
==== Classic Logistic Regression ====

Training Classic Logistic Regression

[Evaluation] Classic Logistic Regression Results:
ROC-AUC:          0.9612
Precision:        0.5229
Recall:           0.1882
F1-Score:         0.2768
Protected Ratio:  28.74%
```

Precision-Recall (PR) Curve for Classic Logistic Regression

```
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc, precision_recall_curve, average_pre

print("\n==== Generating Curves for Classic Logistic Regression ====")

# ==========================================
# 1. Compute ROC curve data and AUC
# ==========================================
fpr_classic, tpr_classic, _ = roc_curve(y_test, y_prob_classic)
roc_auc_classic = auc(fpr_classic, tpr_classic)

# ==========================================
# 2. Compute PR curve data and AP (Average Precision)
# ==========================================
precision_classic, recall_classic, _ = precision_recall_curve(y_test, y_prob_cl
```

```
ap_classic = average_precision_score(y_test, y_prob_classic)

# ==========================================
# 3. Plot the curves
# ==========================================
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# Plot ROC curve (left subplot)
ax1.plot(fpr_classic, tpr_classic, color='green', lw=2,
         label=f'Classic Logistic Regression (AUC = {roc_auc_classic:.4f})')
ax1.plot([0, 1], [0, 1], color='gray', lw=1, linestyle='--')  # Random guess di
ax1.set_xlim([0.0, 1.0])
ax1.set_ylim([0.0, 1.05])
ax1.set_xlabel('False Positive Rate (FPR)')
ax1.set_ylabel('True Positive Rate (TPR)')
ax1.set_title('ROC Curve - Classic Logistic Regression')
ax1.legend(loc="lower right")
ax1.grid(True, alpha=0.3)

# Plot Precision-Recall curve (right subplot)
ax2.plot(recall_classic, precision_classic, color='green', lw=2,
         label=f'Classic Logistic Regression (AP = {ap_classic:.4f})')

# Add baseline for random guessing (true positive ratio in the test set)
baseline_ratio = sum(y_test) / len(y_test)
ax2.axhline(y=baseline_ratio, color='gray', lw=1, linestyle='--',
            label=f'Random Guess ({baseline_ratio:.4f})')

ax2.set_xlim([0.0, 1.0])
ax2.set_ylim([0.0, 1.05])
ax2.set_xlabel('Recall (True Positive Rate)')
ax2.set_ylabel('Precision (Positive Predictive Value)')
ax2.set_title('Precision-Recall (PR) Curve - Classic LR')
ax2.legend(loc="upper right")
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
==== Generating Curves for Classic Logistic Regression ====
```



Optimal Threshold for Classic Logistic Regression

```
print("==== Finding Optimal Threshold for Maximum F1-Score ====\n")

# ==========================================
# 1. Calculate Precision, Recall, and Thresholds
# ==========================================
# precision_recall_curve returns arrays of precision, recall, and thresholds
precisions, recalls, thresholds = precision_recall_curve(y_test, y_prob_classic

# ==========================================
# 2. Calculate F1-Score for every threshold
# ==========================================
# The length of precisions and recalls is len(thresholds) + 1.
# We slice [:-1] to match the length of thresholds arrays.
# Added a tiny epsilon (1e-10) to prevent division by zero warnings.
epsilon = 1e-10
f1_scores = 2 * (precisions[:-1] * recalls[:-1]) / (precisions[:-1] + recalls[:

# ==========================================
# 3. Identify the Best Threshold
# ==========================================
best_idx = np.argmax(f1_scores)
best_threshold = thresholds[best_idx]
best_f1 = f1_scores[best_idx]
```

```
print(f"Search Completed!")
print(f"Optimal Threshold found: {best_threshold:.4f}")
print(f"Maximum F1-Score:        {best_f1:.4f}\n")

# ==========================================
# 4. Evaluate Classic LR with Optimal Threshold
# ==========================================
print("Applying the new optimal threshold to Test Set predictions...")
y_pred_optimal = (y_prob_classic >= best_threshold).astype(int)

# Calculate financial impact metrics with the new predictions
tp_mask_optimal = (y_pred_optimal == 1) & (y_test == 1)
protected_amt_optimal = np.sum(amt_test_full[tp_mask_optimal])
protected_ratio_optimal = (protected_amt_optimal / total_fraud_amt) * 100 if tc

print("\n[Evaluation] Classic LR Results (Optimal Threshold):")
# Note: ROC-AUC evaluates the ranking probabilities and does not change with th
print(f"ROC-AUC:          {roc_auc_score(y_test, y_prob_classic):.4f}")
print(f"Precision:        {precision_score(y_test, y_pred_optimal):.4f}")
print(f"Recall:           {recall_score(y_test, y_pred_optimal):.4f}")
print(f"F1-Score:         {f1_score(y_test, y_pred_optimal):.4f}")
print(f"Protected Amount: ${protected_amt_optimal:,.2f}")
print(f"Protected Ratio:  {protected_ratio_optimal:.2f}%")
```

```
==== Finding Optimal Threshold for Maximum F1-Score ====

Search Completed!
Optimal Threshold found: 0.1064
Maximum F1-Score:        0.4657

Applying the new optimal threshold to Test Set predictions...

[Evaluation] Classic LR Results (Optimal Threshold):
ROC-AUC:          0.9612
Precision:        0.4461
Recall:           0.4871
F1-Score:         0.4657
Protected Amount: $142,801.26
Protected Ratio:  65.07%
```

Cost-Sensitive Bayesian Logistic Regression model

```
tfd = tfp.distributions

print("==== Cost-Sensitive Bayesian HMC via TensorFlow Probability ====\n")

# ==========================================
# Step 1: Data Preparation & Feature Extraction
```

```
# ==========================================
# Extract the specified absolute core features
pruned_production_features = ['hour_cos', 'job_encoded', 'merchant_encoded',
                              'amt_sum_24h', 'amt', 'tx_count_24h']

print(f"Features pruned! Final production features ({len(pruned_production_feat

# Extract features and labels from the pre-engineered train/test sets (preventi
X_train_raw = train_df_engineered[pruned_production_features].copy()
y_train_final = train_df_engineered['is_fraud'].values
amt_train_full = train_df_engineered['amt'].values

X_test_raw = test_df_engineered[pruned_production_features].copy()
y_test_final = test_df_engineered['is_fraud'].values
amt_test_full = test_df_engineered['amt'].values

# Standardize (fit only on training set to prevent data leakage into the test s
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_raw)
X_test_scaled = scaler.transform(X_test_raw)

# Add constant term (intercept) for subsequent models
X_train_const = sm.add_constant(X_train_scaled)
X_test_const = sm.add_constant(X_test_scaled)

# ==========================================
# Step 2: Prepare Cost-Sensitive HMC Samples (TFP)
# ==========================================
# Stratified downsampling for training data to prevent TFP HMC Out-of-Memory (O
print("\nStratified downsampling training data for TFP HMC...")
train_df_for_sampling = pd.DataFrame(X_train_scaled, columns=pruned_production_
train_df_for_sampling['is_fraud'] = y_train_final
train_df_for_sampling['amt'] = amt_train_full

# Extract normal and fraud samples, capping at a maximum of 2500 records each
sample_indices = train_df_for_sampling.groupby('is_fraud', group_keys=False).ap
    lambda x: x.sample(n=min(len(x), 2500), random_state=42),
    include_groups=False
).index

# Shuffle the data
np.random.seed(42)
sample_indices = np.random.permutation(sample_indices)

X_sample_scaled = X_train_scaled[sample_indices]
y_sample = y_train_final[sample_indices]
amt_sample = amt_train_full[sample_indices]

# Add constant term
X_sample_const_hmc = sm.add_constant(X_sample_scaled)
```

```
print("Generating cost-sensitive weight matrix for HMC...")
cost_weights_hmc = np.where(
    y_sample == 1,
    5.0 + np.log1p(amt_sample),
    1.0
)

# Convert to TensorFlow Tensors
X_tf = tf.cast(X_sample_const_hmc, dtype=tf.float32)
y_tf = tf.cast(y_sample, dtype=tf.float32)
weights_tf = tf.cast(cost_weights_hmc, dtype=tf.float32)
num_features = X_tf.shape[1]

# ==========================================
# Step 3: Bayesian HMC with Cost-Sensitive Likelihood
# ==========================================
def log_prob_fn_cost_sensitive_final(coeffs):
    # Prior distribution: Standard Normal
    prior = tf.reduce_sum(tfd.Normal(loc=0., scale=2.).log_prob(coeffs))

    # Calculate Logits (X * weights)
    logits = tf.matmul(X_tf, tf.reshape(coeffs, [-1, 1]))

    # Core logic: Unweighted log-likelihoods (Bernoulli distribution)
    unweighted_log_likelihoods = tfd.Bernoulli(logits=tf.squeeze(logits)).log_p

    # Multiply standard log-likelihoods by our custom cost weights
    weighted_likelihood = tf.reduce_sum(weights_tf * unweighted_log_likelihoods

    return prior + weighted_likelihood

# Configure NUTS sampler and adaptive step size
nuts_kernel = tfp.mcmc.NoUTurnSampler(target_log_prob_fn=log_prob_fn_cost_sensi
adaptive_nuts = tfp.mcmc.DualAveragingStepSizeAdaptation(
    inner_kernel=nuts_kernel, num_adaptation_steps=400, target_accept_prob=0.8
)

@tf.function(autograph=False)
def run_cost_sensitive_chain_final():
    return tfp.mcmc.sample_chain(
        num_results=2000,
        num_burnin_steps=1000,
        current_state=tf.zeros([num_features]),
        kernel=adaptive_nuts,
        trace_fn=lambda _, pkr: pkr.inner_results.is_accepted
    )

print("\nLaunching Ultimate Cost-Sensitive HMC Sampling via TFP (This may take
samples, is_accepted = run_cost_sensitive_chain_final()

# Parse posterior results
```

```
post_samples = samples.numpy()
feature_names_final_plot = ['const'] + pruned_production_features

bayesian_summary_cost_final = pd.DataFrame({
    'Feature': feature_names_final_plot,
    'Posterior Mean': np.mean(post_samples, axis=0),
    'Lower 89% CI': np.percentile(post_samples, 5.5, axis=0),
    'Upper 89% CI': np.percentile(post_samples, 94.5, axis=0)
})

print(f"Sampling completed! Acceptance Rate: {tf.reduce_mean(tf.cast(is_accepte
print("\nCost-Sensitive Bayesian Posterior Distribution Table:")
print(bayesian_summary_cost_final)

# ==========================================
# Step 4: Test Set Evaluation via Bayesian Posterior Means
# ==========================================
print("\nEvaluating Bayesian Posteriors on Independent Test Set...")
# Use posterior means as final model weights
bayesian_weights = bayesian_summary_cost_final['Posterior Mean'].values

# Manually calculate Logits and predicted probabilities on Test Set
test_logits = np.dot(X_test_const, bayesian_weights)
test_probs = 1 / (1 + np.exp(-test_logits)) # Sigmoid function

# Evaluate using default threshold
y_pred_bayesian = (test_probs >= 0.5).astype(int)

# Calculate Financial Impact Metrics for Bayesian Model
total_fraud_mask = (y_test_final == 1)
total_fraud_amt = np.sum(amt_test_full[total_fraud_mask])

tp_mask_bayesian = (y_pred_bayesian == 1) & (y_test_final == 1)
protected_amt_bayesian = np.sum(amt_test_full[tp_mask_bayesian])
protected_ratio_bayesian = (protected_amt_bayesian / total_fraud_amt) * 100 if

print(f"[TFP Bayesian Model] Results on Test Set:")
print(f"ROC-AUC:          {roc_auc_score(y_test_final, test_probs):.4f}")
print(f"Precision:        {precision_score(y_test_final, y_pred_bayesian):.4f}"
print(f"Recall:           {recall_score(y_test_final, y_pred_bayesian):.4f}")
print(f"F1-Score:         {f1_score(y_test_final, y_pred_bayesian):.4f}")
print(f"Protected Amount: ${protected_amt_bayesian:,.2f}")
print(f"Protected Ratio:  {protected_ratio_bayesian:.2f}%")
```

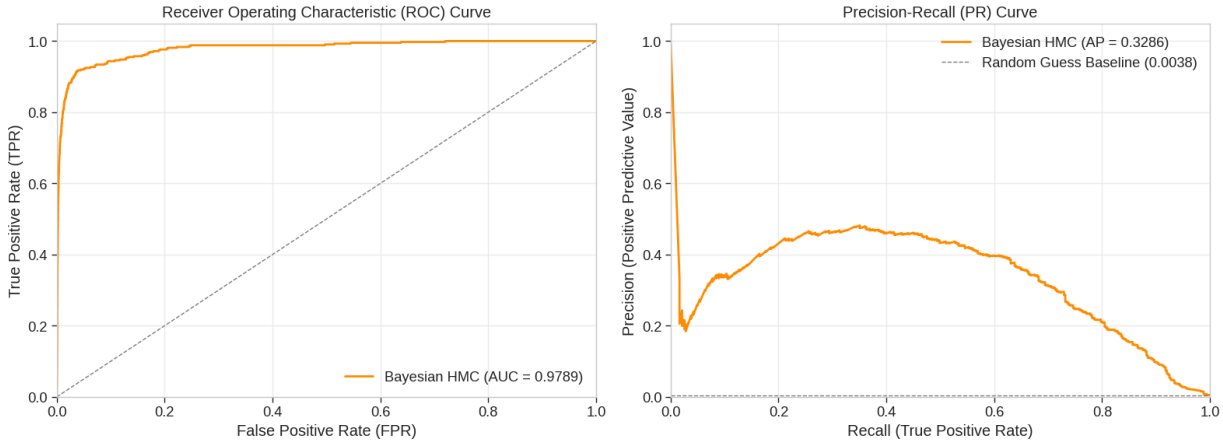
```
==== Cost-Sensitive Bayesian HMC via TensorFlow Probability ====

Features pruned! Final production features (6 dims): ['hour_cos', 'job_encoded',

Stratified downsampling training data for TFP HMC...
Generating cost-sensitive weight matrix for HMC...

Launching Ultimate Cost-Sensitive HMC Sampling via TFP (This may take a moment).
```

```
Sampling completed! Acceptance Rate: 0.9945

Cost-Sensitive Bayesian Posterior Distribution Table:
            Feature  Posterior Mean  Lower 89% CI  Upper 89% CI
0             const       -1.583625     -1.676425     -1.494686
1          hour_cos        1.126951      1.058200      1.196066
2       job_encoded        1.431133      1.328699      1.534580
3  merchant_encoded        0.351398      0.279674      0.425365
4       amt_sum_24h        1.217210      1.146705      1.291737
5               amt        0.651074      0.574196      0.728411
6      tx_count_24h       -0.927362     -1.000237     -0.857131

Evaluating Bayesian Posteriors on Independent Test Set...
[TFP Bayesian Model] Results on Test Set:
ROC-AUC:          0.9789
Precision:        0.0227
Recall:           0.9600
F1-Score:         0.0443
Protected Amount: $218,623.74
Protected Ratio:  99.62%
```

```
print("\n==== Generating Performance Curves for Bayesian HMC ====")

# ==========================================
# 1. Calculate ROC Curve and AUC
# ==========================================
# y_test_final contains the true labels, test_probs contains the Bayesian predi
fpr_bayes, tpr_bayes, _ = roc_curve(y_test_final, test_probs)
roc_auc_bayes = auc(fpr_bayes, tpr_bayes)

# ==========================================
# 2. Calculate Precision-Recall Curve and AP
# ==========================================
# AP (Average Precision) summarizes the PR curve as the weighted mean of precis
precision_bayes, recall_bayes, _ = precision_recall_curve(y_test_final, test_pr
ap_bayes = average_precision_score(y_test_final, test_probs)

# ==========================================
# 3. Plotting the Curves
# ==========================================
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# --- Left Plot: ROC Curve ---
ax1.plot(fpr_bayes, tpr_bayes, color='darkorange', lw=2,
         label=f'Bayesian HMC (AUC = {roc_auc_bayes:.4f})')
# Plot the random guess diagonal line
ax1.plot([0, 1], [0, 1], color='gray', lw=1, linestyle='--')

ax1.set_xlim([0.0, 1.0])
ax1.set_ylim([0.0, 1.05])
ax1.set_xlabel('False Positive Rate (FPR)')
```

```
ax1.set_ylabel('True Positive Rate (TPR)')
ax1.set_title('Receiver Operating Characteristic (ROC) Curve')
ax1.legend(loc="lower right")
ax1.grid(True, alpha=0.3)

# --- Right Plot: Precision-Recall (PR) Curve ---
ax2.plot(recall_bayes, precision_bayes, color='darkorange', lw=2,
         label=f'Bayesian HMC (AP = {ap_bayes:.4f})')

# Calculate the baseline for PR curve (ratio of positive samples in the test se
baseline_ratio = sum(y_test_final) / len(y_test_final)
ax2.axhline(y=baseline_ratio, color='gray', lw=1, linestyle='--',
            label=f'Random Guess Baseline ({baseline_ratio:.4f})')

ax2.set_xlim([0.0, 1.0])
ax2.set_ylim([0.0, 1.05])
ax2.set_xlabel('Recall (True Positive Rate)')
ax2.set_ylabel('Precision (Positive Predictive Value)')
ax2.set_title('Precision-Recall (PR) Curve')
ax2.legend(loc="upper right")
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
==== Generating Performance Curves for Bayesian HMC ====
```




```
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.metrics import precision_recall_curve, precision_score, recall_sco

print("==== Finding Optimal Threshold for Maximum F1-Score (Bayesian HMC) ====\

# ==========================================
# 1. Calculate Precision, Recall, and Thresholds
# ==========================================
# precision_recall_curve returns arrays of precision, recall, and thresholds
precisions, recalls, thresholds = precision_recall_curve(y_test_final, test_prc

# ==========================================
# 2. Calculate F1-Score for every threshold
# ==========================================
# The length of precisions and recalls is len(thresholds) + 1.
# We slice [:-1] to match the length of thresholds arrays.
# Added a tiny epsilon (1e-10) to prevent division by zero warnings.
epsilon = 1e-10
f1_scores = 2 * (precisions[:-1] * recalls[:-1]) / (precisions[:-1] + recalls[:

# ==========================================
# 3. Identify the Best Threshold
# ==========================================
best_idx = np.argmax(f1_scores)
best_threshold = thresholds[best_idx]
best_f1 = f1_scores[best_idx]

print(f"Search Completed!")
print(f"Optimal Threshold found: {best_threshold:.4f}")
print(f"Maximum F1-Score:        {best_f1:.4f}\n")

# ==========================================
# 4. Evaluate Bayesian HMC with Optimal Threshold
# ==========================================
print("Applying the new optimal threshold to Test Set predictions...")
y_pred_optimal_bayes = (test_probs >= best_threshold).astype(int)

# Calculate financial impact metrics with the new predictions
tp_mask_optimal_bayes = (y_pred_optimal_bayes == 1) & (y_test_final == 1)
protected_amt_optimal_bayes = np.sum(amt_test_full[tp_mask_optimal_bayes])
protected_ratio_optimal_bayes = (protected_amt_optimal_bayes / total_fraud_amt)

print("\n[Evaluation] Bayesian HMC Results (Optimal Threshold):")
# Note: ROC-AUC evaluates the ranking probabilities and does not change with th
print(f"ROC-AUC:          {roc_auc_score(y_test_final, test_probs):.4f}")
print(f"Precision:        {precision_score(y_test_final, y_pred_optimal_bayes):
print(f"Recall:           {recall_score(y_test_final, y_pred_optimal_bayes):.4f
print(f"F1-Score:         {f1_score(y_test_final, y_pred_optimal_bayes):.4f}")
print(f"Protected Amount: ${protected_amt_optimal_bayes:,.2f}")
print(f"Protected Ratio:  {protected_ratio_optimal_bayes:.2f}%")
```

```
==== Finding Optimal Threshold for Maximum F1-Score (Bayesian HMC) ====

Search Completed!
Optimal Threshold found: 0.9988
Maximum F1-Score:        0.4821

Applying the new optimal threshold to Test Set predictions...

[Evaluation] Bayesian HMC Results (Optimal Threshold):
ROC-AUC:          0.9789
Precision:        0.3958
Recall:           0.6165
F1-Score:         0.4821
Protected Amount: $182,864.36
Protected Ratio:  83.33%
```

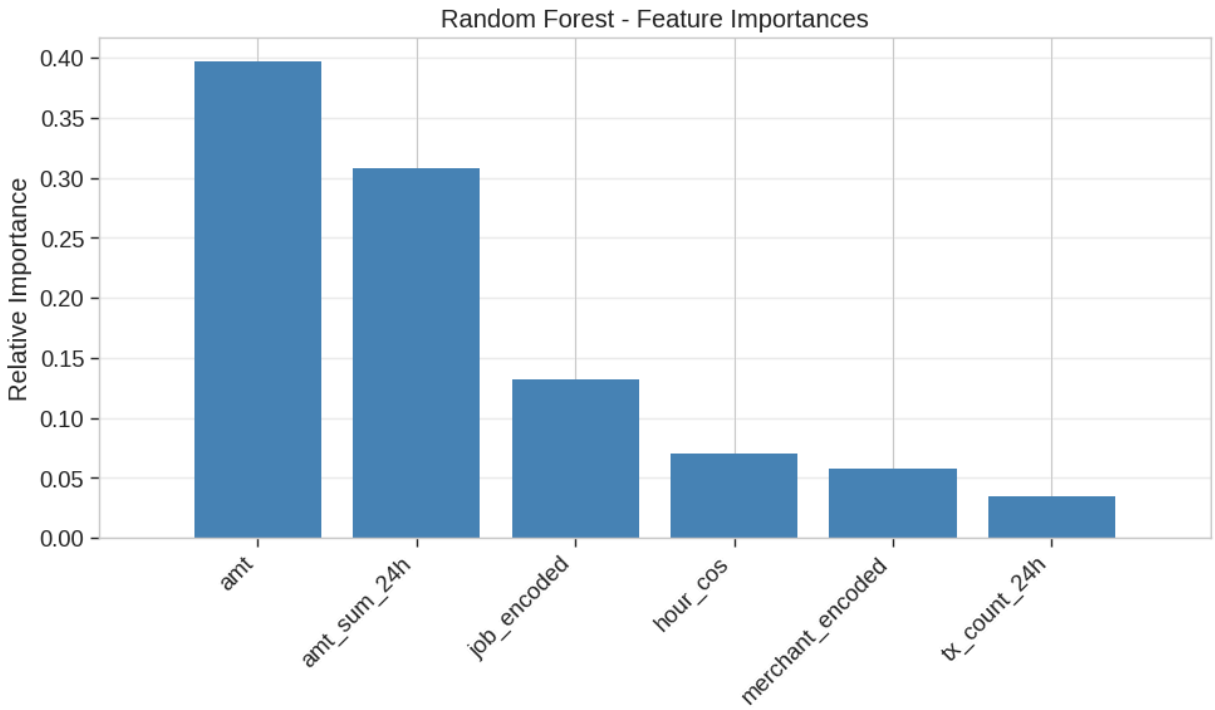
Cost-Sensitive Random Forest

```
print("\n==== Stage 6: Cost-Sensitive Random Forest ====\n")

# ==========================================
# 1. Prepare the cost-sensitive weight matrix
# ==========================================
print("Generating cost-sensitive weight matrix for Random Forest...")

# Use the adjusted sensitivity weights from the previous stage
# Fraud samples receive higher weights, especially for large transaction amount
cost_weights_rf = np.where(
    y_train_final == 1,
    5.0 + np.log1p(amt_train_full),
    1.0
)

# ==========================================
# 2. Train Random Forest model
# ==========================================
# Note: Random Forest does not require an intercept term
# and does not strictly require feature standardization,
# so we directly use the raw feature data
print("Training Cost-Sensitive Random Forest (This might take a moment)...")

rf_model = RandomForestClassifier(
    n_estimators=150,       # Number of trees in the forest
    max_depth=12,           # Limit tree depth to prevent overfitting on highly
    min_samples_leaf=5,     # Minimum samples per leaf node to improve robustne
    random_state=42,
    n_jobs=-1               # Use all CPU cores for faster training
)
```

```
# Train the model with sample weights to emphasize high-value fraud cases
rf_model.fit(X_train_raw, y_train_final, sample_weight=cost_weights_rf)

# ==========================================
# 3. Evaluate on the test set
# ==========================================
print("\nEvaluating Random Forest on Independent Test Set...")

# Get predicted probabilities for the positive class (fraud)
y_prob_rf = rf_model.predict_proba(X_test_raw)[:, 1]

# Use the same custom threshold as before for fair comparison
RF_CUSTOM_THRESHOLD = 0.3
y_pred_rf = (y_prob_rf >= RF_CUSTOM_THRESHOLD).astype(int)

# Compute financial impact metrics (fraud interception effectiveness)
total_fraud_mask = (y_test_final == 1)
total_fraud_amt = np.sum(amt_test_full[total_fraud_mask])

# True positives: correctly identified fraud transactions
tp_mask_rf = (y_pred_rf == 1) & (y_test_final == 1)

# Total amount of fraud successfully prevented
protected_amt_rf = np.sum(amt_test_full[tp_mask_rf])

# Percentage of total fraud amount that was successfully protected
protected_ratio_rf = (protected_amt_rf / total_fraud_amt) * 100 if total_fraud_

print(f"[Random Forest] Results on Test Set:")
print(f"ROC-AUC:          {roc_auc_score(y_test_final, y_prob_rf):.4f}")
print(f"Precision:        {precision_score(y_test_final, y_pred_rf):.4f}")
print(f"Recall:           {recall_score(y_test_final, y_pred_rf):.4f}")
print(f"F1-Score:         {f1_score(y_test_final, y_pred_rf):.4f}")
print(f"Protected Amount: ${protected_amt_rf:,.2f}")
print(f"Protected Ratio:  {protected_ratio_rf:.2f}%")

# ==========================================
# 4. Plot Feature Importance
# ==========================================
importances = rf_model.feature_importances_
indices = np.argsort(importances)[::-1]
sorted_features = [pruned_production_features[i] for i in indices]

plt.figure(figsize=(10, 6))
plt.title("Random Forest - Feature Importances")

# Plot feature importance values in descending order
plt.bar(range(len(importances)), importances[indices], color="steelblue", aligr

# Display feature names on x-axis
plt.xticks(range(len(importances)), sorted_features, rotation=45, ha='right')
```

```
plt.xlim([-1, len(importances)])
plt.ylabel("Relative Importance")
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

```
==== Stage 6: Cost-Sensitive Random Forest ====

Generating cost-sensitive weight matrix for Random Forest...
Training Cost-Sensitive Random Forest (This might take a moment)...

Evaluating Random Forest on Independent Test Set...
[Random Forest] Results on Test Set:
ROC-AUC:          0.9914
Precision:        0.6281
Recall:           0.7671
F1-Score:         0.6907
Protected Amount: $202,436.33
Protected Ratio:  92.25%
```

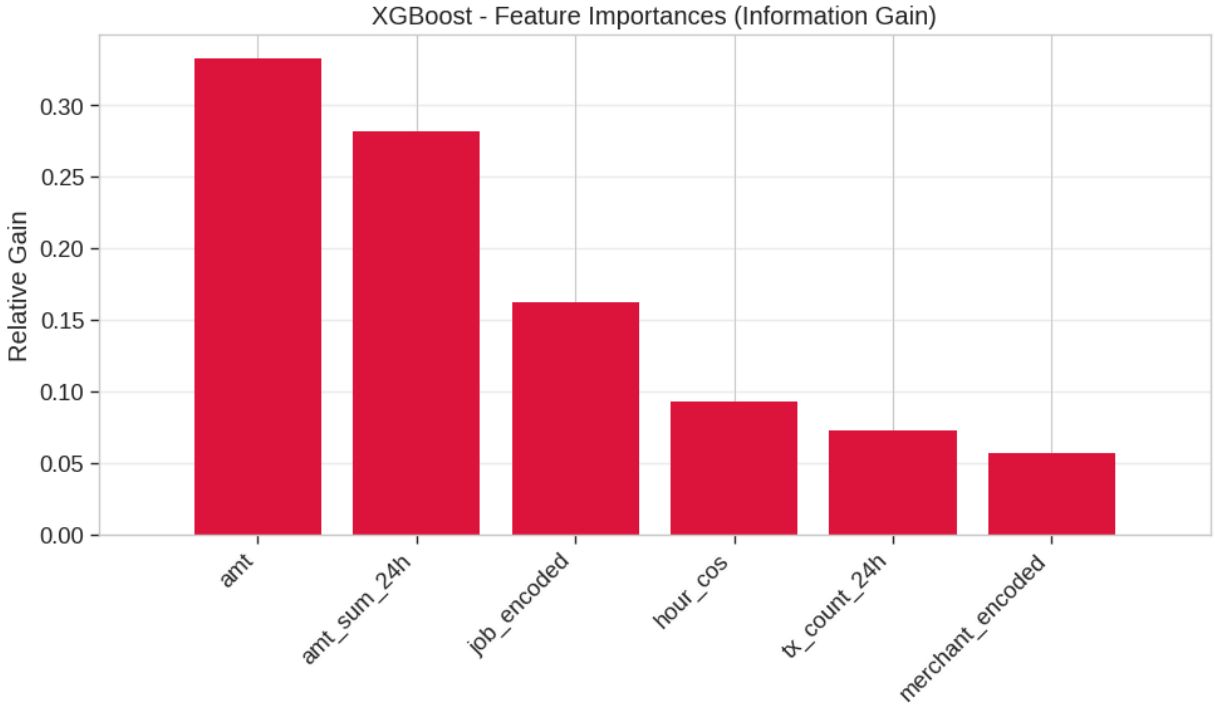


Cost-Sensitive XGBoost

```
print("\n==== Stage 7: Cost-Sensitive XGBoost (Gradient Boosting) ====\n")

# ==========================================
# 1. Prepare Cost-Sensitive Weights
# ==========================================
print("Generating cost-sensitive weight matrix for XGBoost...")

# Use the adjusted smooth weighting scheme: 5.0 + ln(1 + amount)
# Fraud samples receive higher weights, especially for large transactions
cost_weights_xgb = np.where(
    y_train_final == 1,
    5.0 + np.log1p(amt_train_full),
    1.0
)

# ==========================================
# 2. Train Cost-Sensitive XGBoost Model
# ==========================================
print("Training Cost-Sensitive XGBoost Model (Learning from mistakes)...")

# XGBoost provides more hyperparameters to control overfitting,
# which is especially important for highly imbalanced datasets
xgb_model = xgb.XGBClassifier(
    n_estimators=200,          # Number of trees (boosting rounds)
    max_depth=5,               # Tree depth (shallower trees reduce overfitting
    learning_rate=0.05,        # Learning rate (smaller step size improves conv
    subsample=0.8,             # Use 80% of samples for each tree (random sampl
    colsample_bytree=0.8,      # Use 80% of features for each tree
    random_state=42,
    n_jobs=-1,                 # Use all CPU cores
    eval_metric='logloss'      # Explicit evaluation metric to avoid warnings
)

# Train model using raw features and sample weights
xgb_model.fit(X_train_raw, y_train_final, sample_weight=cost_weights_xgb)

# ==========================================
# 3. Evaluate on Independent Test Set
# ==========================================
print("\nEvaluating XGBoost on Test Set...")

# Get predicted probabilities for fraud class
y_prob_xgb = xgb_model.predict_proba(X_test_raw)[:, 1]

# Apply custom decision threshold
XGB_CUSTOM_THRESHOLD = 0.3
y_pred_xgb = (y_prob_xgb >= XGB_CUSTOM_THRESHOLD).astype(int)
```

```
# Recompute financial impact metrics (ensure no variable contamination)
total_fraud_mask = (y_test_final == 1)
total_fraud_amt = np.sum(amt_test_full[total_fraud_mask])

# True positives: correctly detected fraud transactions
tp_mask_xgb = (y_pred_xgb == 1) & (y_test_final == 1)

# Total amount of fraud successfully prevented
protected_amt_xgb = np.sum(amt_test_full[tp_mask_xgb])

# Percentage of fraud amount successfully protected
protected_ratio_xgb = (protected_amt_xgb / total_fraud_amt) * 100 if total_frau

print(f"[XGBoost] Results on Test Set:")
print(f"ROC-AUC:          {roc_auc_score(y_test_final, y_prob_xgb):.4f}")
print(f"Precision:        {precision_score(y_test_final, y_pred_xgb):.4f}")
print(f"Recall:           {recall_score(y_test_final, y_pred_xgb):.4f}")
print(f"F1-Score:         {f1_score(y_test_final, y_pred_xgb):.4f}")
print(f"Protected Amount: ${protected_amt_xgb:,.2f}")
print(f"Protected Ratio:  {protected_ratio_xgb:.2f}%")

# ==========================================
# 4. Plot Feature Importance (Gain)
# ==========================================
# XGBoost's default importance type is "gain",
# which measures the average information gain contributed by each feature split
importances_xgb = xgb_model.feature_importances_
indices_xgb = np.argsort(importances_xgb)[::-1]
sorted_features_xgb = [pruned_production_features[i] for i in indices_xgb]

plt.figure(figsize=(10, 6))
plt.title("XGBoost - Feature Importances (Information Gain)")

# Plot feature importance in descending order
plt.bar(range(len(importances_xgb)), importances_xgb[indices_xgb], color="crims

# Show feature names on x-axis
plt.xticks(range(len(importances_xgb)), sorted_features_xgb, rotation=45, ha='r

plt.xlim([-1, len(importances_xgb)])
plt.ylabel("Relative Gain")
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

```
==== Stage 7: Cost-Sensitive XGBoost (Gradient Boosting) ====

Generating cost-sensitive weight matrix for XGBoost...
Training Cost-Sensitive XGBoost Model (Learning from mistakes)...

Evaluating XGBoost on Test Set...
[XGBoost] Results on Test Set:
ROC-AUC:          0.9928
Precision:        0.4896
Recall:           0.8306
F1-Score:         0.6161
Protected Amount: $210,175.23
Protected Ratio:  95.77%
```



Bayesian Optimization XGBoost

```
print("\n==== Stage 8: Bayesian Hyperparameter Optimization for Cost-Sensitive
```

```
# ==========================================
# 1. Define the Objective Function for Optuna
# ==========================================
def objective(trial):
    # 1.1 Define the hyperparameter search space (Bayesian priors)
    param_space = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 400),
        'max_depth': trial.suggest_int('max_depth', 3, 9),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=Tr
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.6, 1.0),
        'min_child_weight': trial.suggest_int('min_child_weight', 1, 10),
        'gamma': trial.suggest_float('gamma', 0.0, 5.0),  # Penalizes leaf spli
        'random_state': 42,
        'n_jobs': -1,
        'eval_metric': 'logloss'
    }

    # 1.2 Setup 3-Fold Stratified Cross Validation
    skf = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
    cv_pr_auc_scores = []

    # Reset indices to ensure safe cross-validation splitting
    X_train_cv = X_train_raw.reset_index(drop=True)
    y_train_cv = y_train_final.copy()
    amt_train_cv = amt_train_full.copy()

    for train_idx, val_idx in skf.split(X_train_cv, y_train_cv):
        # Split training and validation folds
        X_tr, X_val = X_train_cv.iloc[train_idx], X_train_cv.iloc[val_idx]
        y_tr, y_val = y_train_cv[train_idx], y_train_cv[val_idx]
        amt_tr = amt_train_cv[train_idx]

        # Apply cost-sensitive weights ONLY to the training fold
        weights_tr = np.where(y_tr == 1, 3.0 + np.log1p(amt_tr), 1.0)

        # Train XGBoost model for this trial
        model = xgb.XGBClassifier(**param_space)
        model.fit(X_tr, y_tr, sample_weight=weights_tr)

        # Predict on validation fold and compute Average Precision (PR-AUC)
        val_probs = model.predict_proba(X_val)[:, 1]
        pr_auc = average_precision_score(y_val, val_probs)
        cv_pr_auc_scores.append(pr_auc)

    # Return mean PR-AUC across all folds (Optuna will maximize this)
    return np.mean(cv_pr_auc_scores)

# ==========================================
# 2. Execute Bayesian Optimization Study
# ==========================================
```

```
print("Launching Optuna Bayesian Optimization (Searching for 30 trials)...")

# Set direction='maximize' because we want to maximize Average Precision
study = optuna.create_study(direction='maximize', sampler=optuna.samplers.TPESa

# n_trials=30 is a trade-off between computation time and performance
# Increase to 50 or 100 if more computational resources are available
study.optimize(objective, n_trials=30)

print("\n[Optimization Complete]")
print(f"Best Trial PR-AUC (Cross-Validation): {study.best_value:.4f}")
print("Best Hyperparameters Discovered:")
for key, value in study.best_params.items():
    print(f"  {key}: {value}")

# ==========================================
# 3. Train Final Model with Best Parameters
# ==========================================
print("\nTraining Final XGBoost Model using Best Parameters on FULL Training Se

best_params = study.best_params
best_params['random_state'] = 42
best_params['n_jobs'] = -1
best_params['eval_metric'] = 'logloss'

final_xgb_model = xgb.XGBClassifier(**best_params)

# Generate cost-sensitive weights for the full training dataset
final_cost_weights_xgb = np.where(
    y_train_final == 1,
    5.0 + np.log1p(amt_train_full),
    1.0
)

final_xgb_model.fit(X_train_raw, y_train_final, sample_weight=final_cost_weight

# ==========================================
# 4. Evaluate Final Optimized Model on Test Set
# ==========================================
print("Evaluating Final Optimized XGBoost on Independent Test Set...")

y_prob_final_xgb = final_xgb_model.predict_proba(X_test_raw)[:, 1]

# Use previous threshold to ensure fair comparison
FINAL_CUSTOM_THRESHOLD = 0.2
y_pred_final_xgb = (y_prob_final_xgb >= FINAL_CUSTOM_THRESHOLD).astype(int)

# Recompute financial fraud interception metrics
total_fraud_mask = (y_test_final == 1)
total_fraud_amt = np.sum(amt_test_full[total_fraud_mask])
```

```
# True positives: correctly detected fraud
tp_mask_final_xgb = (y_pred_final_xgb == 1) & (y_test_final == 1)

# Total protected fraud amount
protected_amt_final_xgb = np.sum(amt_test_full[tp_mask_final_xgb])

# Percentage of fraud amount successfully prevented
protected_ratio_final_xgb = (protected_amt_final_xgb / total_fraud_amt) * 100 i

print(f"\nOptimized Cost-Sensitive XGBoost Results:")
print(f"ROC-AUC:          {roc_auc_score(y_test_final, y_prob_final_xgb):.4f}")
print(f"Precision:        {precision_score(y_test_final, y_pred_final_xgb):.4f}
print(f"Recall:           {recall_score(y_test_final, y_pred_final_xgb):.4f}")
print(f"F1-Score:         {f1_score(y_test_final, y_pred_final_xgb):.4f}")
print(f"Protected Amount: ${protected_amt_final_xgb:,.2f}")
print(f"Protected Ratio:  {protected_ratio_final_xgb:.2f}%")
```

```
[I 2026-04-13 02:12:12,765] A new study created in memory with name: no-name-6

==== Stage 8: Bayesian Hyperparameter Optimization for Cost-Sensitive XGBoost

Launching Optuna Bayesian Optimization (Searching for 30 trials)...
[I 2026-04-13 02:12:51,670] Trial 0 finished with value: 0.8820552581188416 an
[I 2026-04-13 02:13:31,120] Trial 1 finished with value: 0.8761562529236956 an
[I 2026-04-13 02:13:43,886] Trial 2 finished with value: 0.8396536761504079 an
[I 2026-04-13 02:13:56,241] Trial 3 finished with value: 0.8581308875766999 an
[I 2026-04-13 02:14:19,788] Trial 4 finished with value: 0.8643487238961453 an
[I 2026-04-13 02:14:47,988] Trial 5 finished with value: 0.8629723905354582 an
[I 2026-04-13 02:15:01,285] Trial 6 finished with value: 0.8655918283460237 an
[I 2026-04-13 02:15:12,558] Trial 7 finished with value: 0.8736766745233328 an
[I 2026-04-13 02:15:21,820] Trial 8 finished with value: 0.8043482963220447 an
[I 2026-04-13 02:15:40,367] Trial 9 finished with value: 0.8674290965248269 an
[I 2026-04-13 02:16:04,758] Trial 10 finished with value: 0.8765137179733115 a
[I 2026-04-13 02:16:26,880] Trial 11 finished with value: 0.8752214523201612 a
[I 2026-04-13 02:16:50,372] Trial 12 finished with value: 0.871942864434616 an
[I 2026-04-13 02:17:09,088] Trial 13 finished with value: 0.8791504056868717 a
[I 2026-04-13 02:17:27,066] Trial 14 finished with value: 0.8788634317639863 a
[I 2026-04-13 02:17:45,812] Trial 15 finished with value: 0.8772309303221268 a
[I 2026-04-13 02:18:11,156] Trial 16 finished with value: 0.8822995375540019 a
[I 2026-04-13 02:18:39,647] Trial 17 finished with value: 0.8820814054909625 a
[I 2026-04-13 02:19:15,323] Trial 18 finished with value: 0.881998230881106 an
[I 2026-04-13 02:19:40,003] Trial 19 finished with value: 0.8794458870066313 a
[I 2026-04-13 02:20:07,157] Trial 20 finished with value: 0.8645952561519435 a
[I 2026-04-13 02:20:52,618] Trial 21 finished with value: 0.8848672932617125 a
[I 2026-04-13 02:21:24,766] Trial 22 finished with value: 0.8853575711805881 a
[I 2026-04-13 02:21:55,443] Trial 23 finished with value: 0.8840317524974312 a
[I 2026-04-13 02:22:25,973] Trial 24 finished with value: 0.8844689789330311 a
[I 2026-04-13 02:22:57,083] Trial 25 finished with value: 0.8842891887555182 a
[I 2026-04-13 02:23:26,174] Trial 26 finished with value: 0.8830311265342283 a
[I 2026-04-13 02:23:52,567] Trial 27 finished with value: 0.8604305057076224 a
[I 2026-04-13 02:24:17,140] Trial 28 finished with value: 0.8811811488224185 a
[I 2026-04-13 02:24:51,682] Trial 29 finished with value: 0.8777850332566413 a

[Optimization Complete]
```

```
Best Trial PR-AUC (Cross-Validation): 0.8854
Best Hyperparameters Discovered:
  n_estimators: 398
  max_depth: 6
  learning_rate: 0.040134971016697676
  subsample: 0.9248864105731716
  colsample_bytree: 0.8132822891458465
  min_child_weight: 4
  gamma: 1.7926513422209431

Training Final XGBoost Model using Best Parameters on FULL Training Set...
Evaluating Final Optimized XGBoost on Independent Test Set...

Optimized Cost-Sensitive XGBoost Results:
ROC-AUC:          0.9937
Precision:        0.4794
Recall:           0.8494
F1-Score:         0.6129
Protected Amount: $210,448.77
Protected Ratio:  95.00%
```

```
import matplotlib.pyplot as plt
from sklearn.metrics import precision_recall_curve, average_precision_score

# Set academic plotting style
plt.style.use('seaborn-v0_8-whitegrid')
plt.figure(figsize=(11, 7))  # Slightly wider figure to allow space for the leg

# ==========================================
# 1. Plot Baseline Logistic Regression
# ==========================================
precision_lr, recall_lr, _ = precision_recall_curve(y_test, y_prob_classic)
ap_lr = average_precision_score(y_test, y_prob_classic)

plt.plot(recall_lr, precision_lr,
         label=f'Baseline LR (PR-AUC = {ap_lr:.3f})',
         color='#7f8c8d', linestyle=':', linewidth=2)

# ==========================================
# 2. Plot Bayesian HMC (New model)
# ==========================================
# Use test_probs and y_test_final from your Bayesian model output
precision_bayes, recall_bayes, _ = precision_recall_curve(y_test_final, test_pr
ap_bayes = average_precision_score(y_test_final, test_probs)

plt.plot(recall_bayes, precision_bayes,
         label=f'Bayesian HMC (PR-AUC = {ap_bayes:.3f})',
         color='darkorange', linestyle='-.', linewidth=2)

# ==========================================
# 3. Plot Random Forest
# ==========================================
precision_rf, recall_rf, _ = precision_recall_curve(y_test_final, y_prob_rf)
```

```
ap_rf = average_precision_score(y_test_final, y_prob_rf)

plt.plot(recall_rf, precision_rf,
         label=f'Random Forest (PR-AUC = {ap_rf:.3f})',
         color='#2980b9', linestyle='--', linewidth=2)

# ==========================================
# 4. Plot Optuna-Tuned XGBoost
# ==========================================
try:
    precision_xgb, recall_xgb, _ = precision_recall_curve(y_test_final, y_prob_
    ap_xgb = average_precision_score(y_test_final, y_prob_xgb)

    plt.plot(recall_xgb, precision_xgb,
             label=f'Optuna XGBoost (PR-AUC = {ap_xgb:.3f})',
             color='#c0392b', linestyle='-', linewidth=3)

except NameError:
    print("Warning: y_prob_xgb not found. Please ensure the XGBoost predictions

# ==========================================

plt.title('Precision-Recall Curve Evolution Across All Models',
          fontsize=16, fontweight='bold', pad=15)

plt.xlabel('Recall (Detection Rate)', fontsize=14)
plt.ylabel('Precision (Accuracy of Fraud Alerts)', fontsize=14)

plt.legend(loc='lower left', fontsize=12)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])

plt.tight_layout()
plt.savefig('fig_pr_curve_comparison_all.png', dpi=300)
plt.show()

print("Full multi-model PR curve comparison (including Bayesian HMC) has been g
```